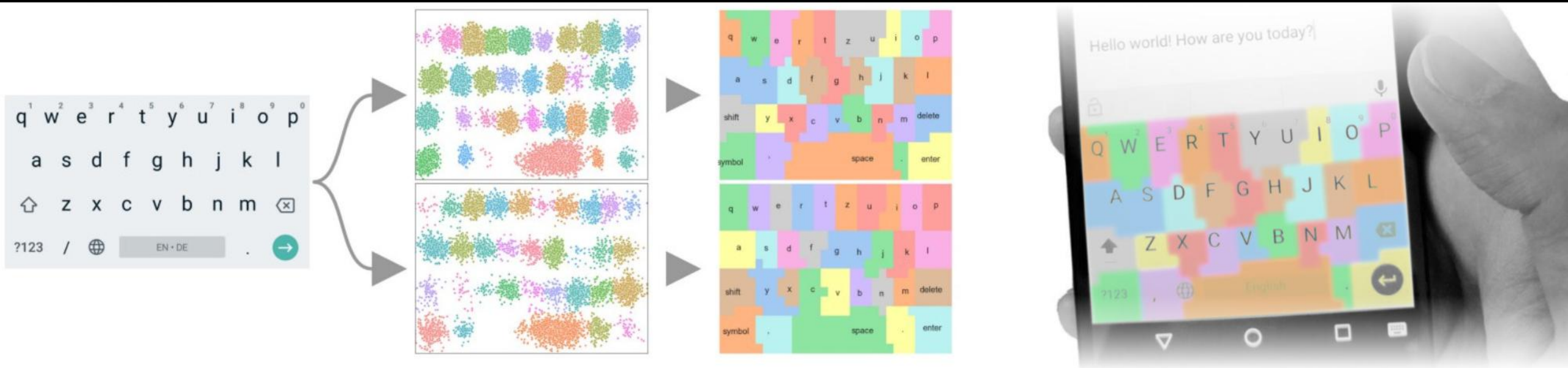


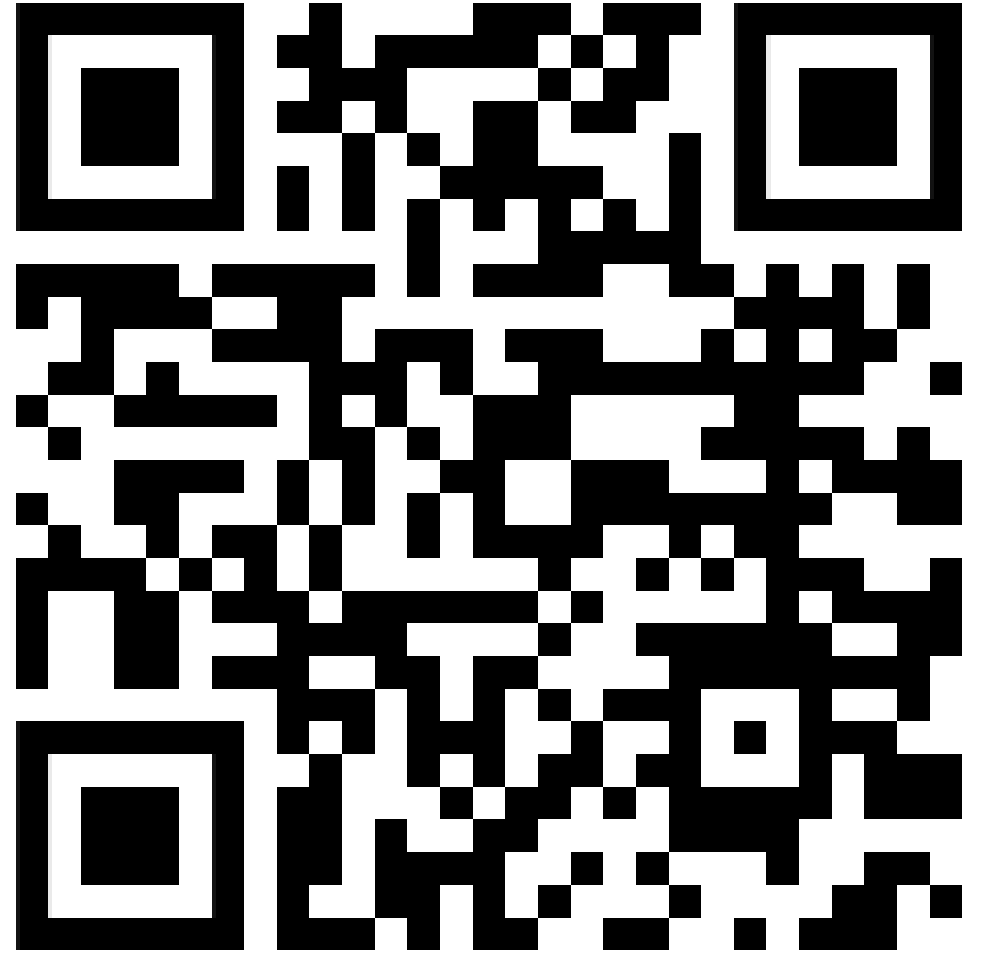


Intelligent Text Entry

If you're here early...

Take a “typing test”!

Press enter
after each word
(not space)!



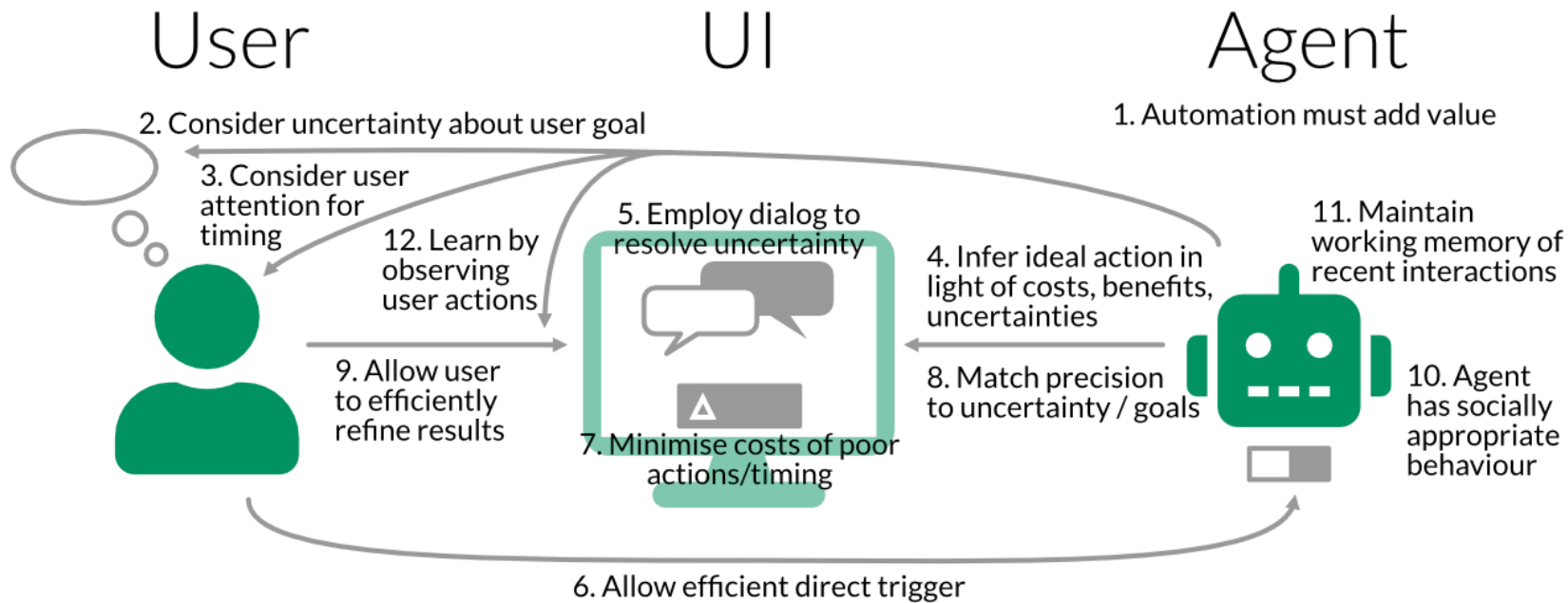
<https://www.livechat.com/typing-speed-test>

Recap & Last Quiz

Lecture Quiz

Think about **Horvitz' twelve principles** in the context of the design of the **word suggestion feature on your smartphone keyboard**:

- To what extent are (some of) these principles realised?
- And could you use them to redesign or improve the word suggestion feature in some way?



Lecture Quiz

Some of your “mappings” and ideas for improvements:

Consider uncertainty about user goals: keyboards usually show **multiple suggestions** because the system is unsure what the user wants.

Ideal action would be the **correct prediction** of the next word

One principle that is not well handled is **minimizing the cost of poor actions**. If you accidentally tap the wrong suggestion, fixing it takes more effort than if you had just typed the word yourself. An improvement would be a brief **undo option** that appears right after a suggestion is accepted, so the user can instantly reverse a wrong tap without retyping.

(7) **Minimizing the cost of poor guesses about action and timing.** Designs for services and alerts should be undertaken with an eye to **minimizing the cost of poor guesses**, including appropriate timing out and natural gestures for rejecting attempts at service.

Note: While the undo is a great idea, that principle originally refers to the cost of poor system actions (e.g. bad suggestions), not a user slip!

Text suggestion feature saves time (so it adds value), learns from user behavior by remembering the word frequency and by that, it has a memory and is aware of the currently-typed text. It however could be improved by **letting users refine the results**. For example, by clicking on the suggested words, a user could get the lexically related words.

To improve the feature, **suggestions could appear only when the user pauses** typing briefly. This would minimize distraction and improve timing, following the principle **"Minimise costs of poor actions/timing"**.

Note: See the next slide for related work that studied if this is a good idea!

If you're curious...

About suggestions and auto-correct

- What do you think is the “role” of text suggestions for your writing process?
Interacting with Next-Phrase Suggestions: How Suggestion Systems Aid and Influence the Cognitive Processes of Writing
- **Timing** of suggestions – is it a good idea to always show suggestions?
A Cost-Benefit Study of Text Entry Suggestion Interaction
- **Autocorrect** – could you benefit from relying more on it?
The Limits of Expert Text Entry Speed on Mobile Keyboards with Autocorrect
- Word **suggestions can slow you down** – why are they still used?
Typing Behavior is About More than Speed: Users' Strategies for Choosing Word Suggestions Despite Slower Typing Rates

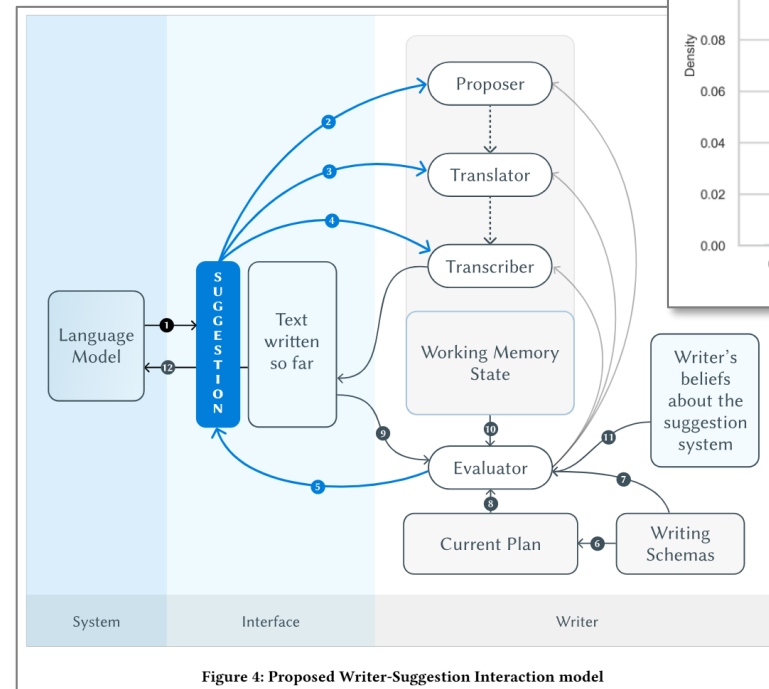
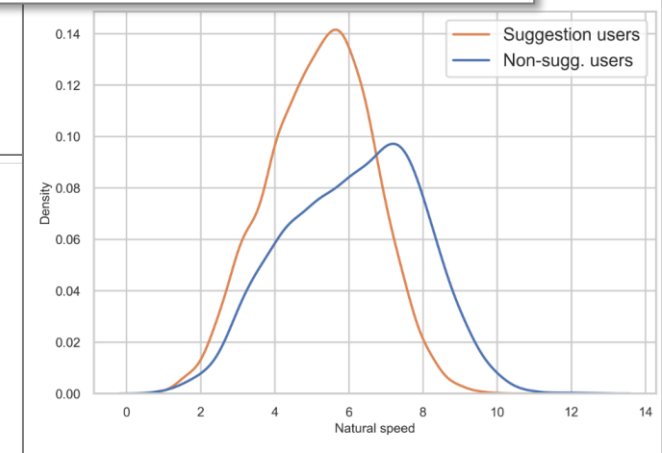
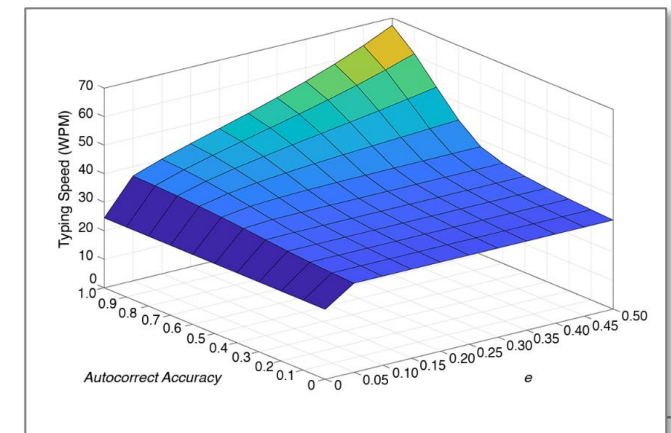
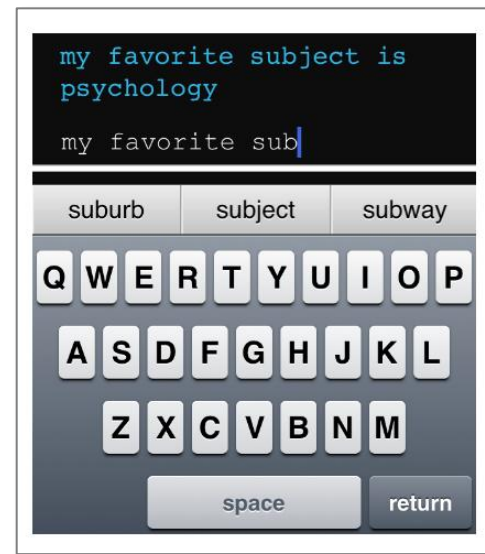


Figure 4: Proposed Writer-Suggestion Interaction model

Lecture Quiz

Apply the framework by Guzdial & Riedl to examine the moodboard system by Koch et al.



Aspect	User	System
Start	Adds first image to the moodboard	None (or: suggests image, if we assume it can suggest sth for an empty board)
Artifact actions	Adding/editing images on the board	None (the system cannot directly edit the board)
Other actions	Trigger new recommendations (i.e. the 3 buttons)	Recommends an image (+ explanation)
Non-turn actions	Not so clear, possibly: Read explanation	Possibly: Learn from user actions
Turns	End after pressing one of the buttons or adding a picture (assuming that the AI updates its recommendation after each board change)	End after displaying a new suggestion
End	User decides (e.g. satisfied with board)	-

This lecture

Learning goals

After this session, you should be able to:

- ✓ **Explain** the motivations and limitations of keyboard adaptation
- ✓ **Explain** how keyboard adaptation, word suggestions, and auto-correction are conceptually related
- ✓ **Analyse** a GUI to give ideas for how it could be modelled probabilistically
(e.g. for adaptation and/or combination with other info, e.g. from a language model)
- ✓ **Explain** beam pruning
- ✓ **Conduct** sequence decoding with token passing
(e.g. by sketching a visualization of the steps of the algorithm for a simple (!) example, as in this lecture)



Intro

Motivation

Own experiences

Background

Why study text entry?

In general in HCI and for IUI specifically

- **High practical relevance:** Everyone types every day
- **Challenging interaction**
 - Very fast interaction loop
 - Requires learning effort (by the user)
- **Technically challenging**
 - E.g. interpreting imprecise typing
 - Vast search space (all possible words/sentences)
- **As a „case study“**
 - Clear boundaries despite these challenges
 - Many ideas in research, popular problem for new ideas
 - Exemplar of sequential input



Why work on text entry?

- „Inviscid entry rate“:
Bottleneck is not the text entry UI but coming up with the text
- Estimated as 67 WPM

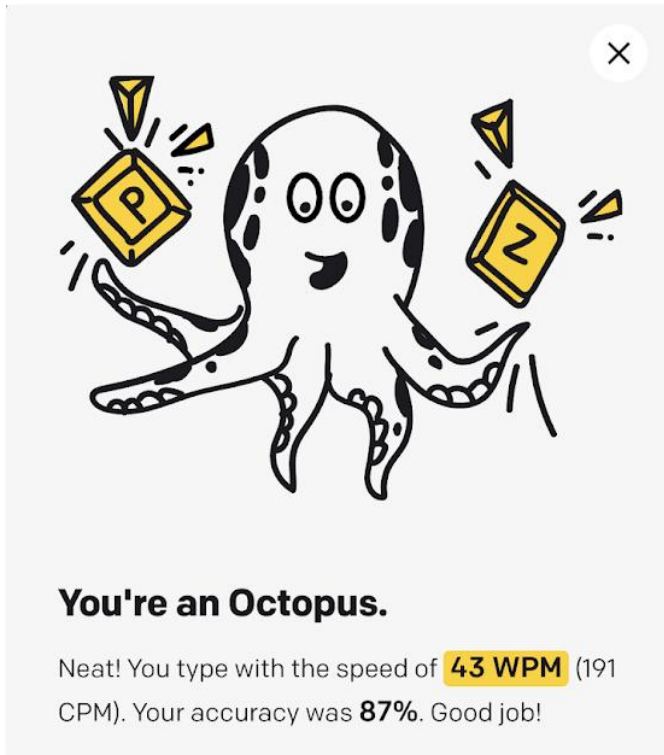
→ Try to reach this on your phone without errors, e.g. in an online typing speed test.

Text entry method	Highest reported entry rate (wpm)
<i>Estimate of the inviscid entry rate</i>	67
Physical thumb keyboards	60 [3]
Gesture keyboards	45 [9]
Optimized on-screen keyboards	45 [12]
QWERTY on-screen keyboards	40 [12]
KALQ thumb keyboard	37 [14]
Half-QWERTY	35 [13]
Twiddler	35 [11]
WalkType	31 [5]
ContextType	28 [6]
Disambiguating keypads	26 [7]
Unconstrained handwriting recognition	25 [8]
Dasher	20 [21]
Mobile speech	18 [18]
Quikwriting	16 [15]
Unistrokes	16 [1]
TiltText	14 [22]
Multi-tap	12 [23]
Graffiti	11 [1]
EdgeWrite	7 [24]

[Kristensson and Vertanen 2014]

How fast can you type?

Let's do a "typing test"



Press enter
after each word
(not space)!



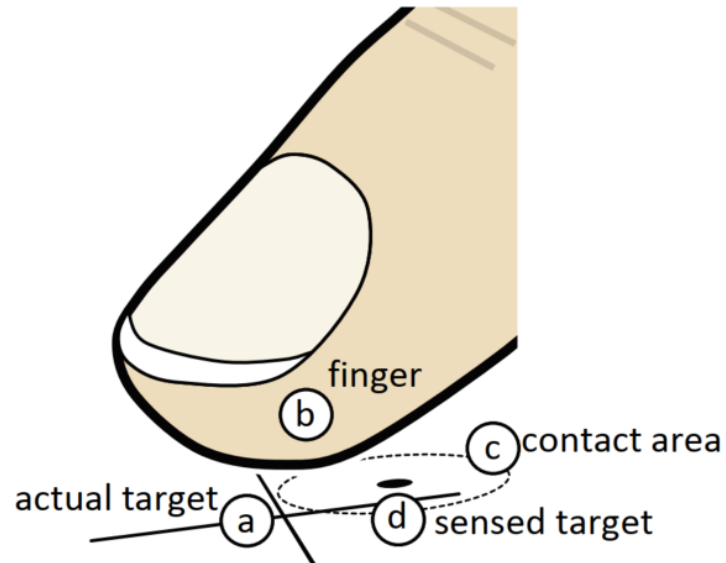
<https://www.livechat.com/typing-speed-test>

Challenges for mobile typing

Some reasons for why we don't always hit keys “pixel perfectly”

Parallax

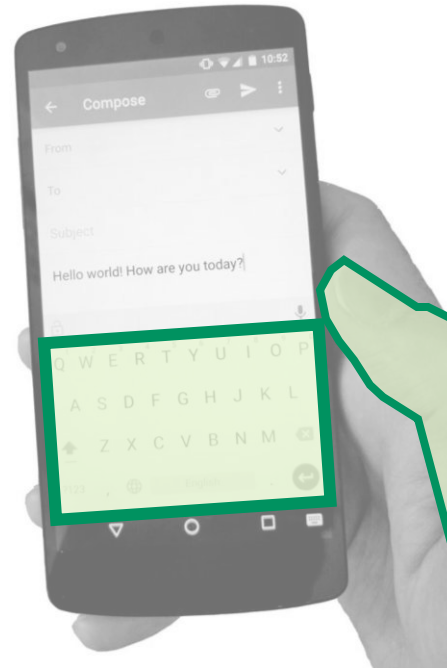
eye – finger - screen



[Holz and Baudisch 2011]

Mobile use

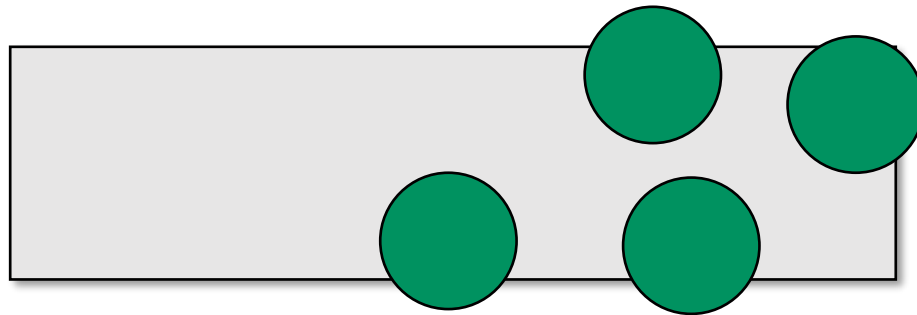
1-2 fingers, small keys, body movement



Variance in touchscreen keypresses

Around a given target GUI element

- Targeting errors also mean that touch locations vary for the same target GUI element
- E.g. hitting a button multiple times:



*What's a good model
of the distribution of touch points?*

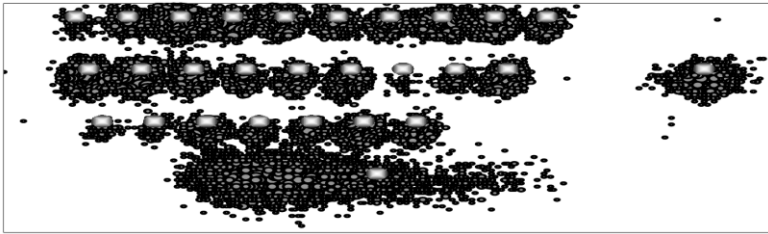


*Collect data
to find out!*

Variance in touchscreen keypresses

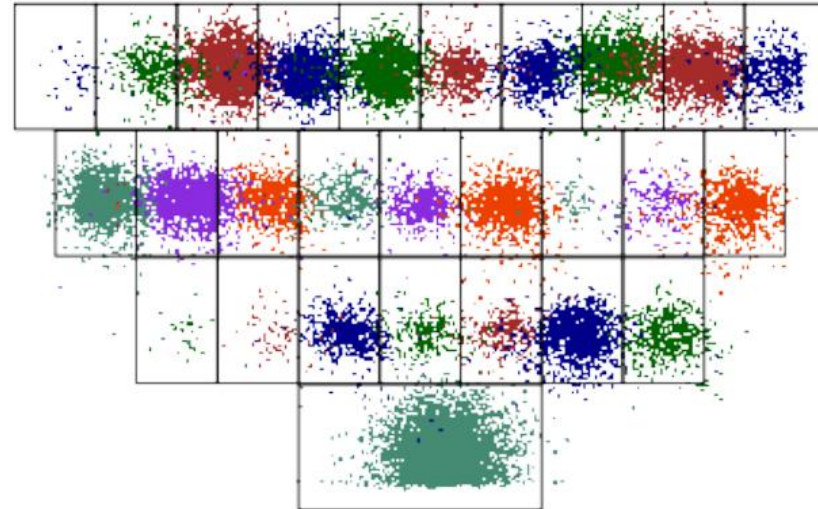
Spread of x,y touch locations around key centres (cf. previous lecture)

PDA



[Goodman et al. 2002]

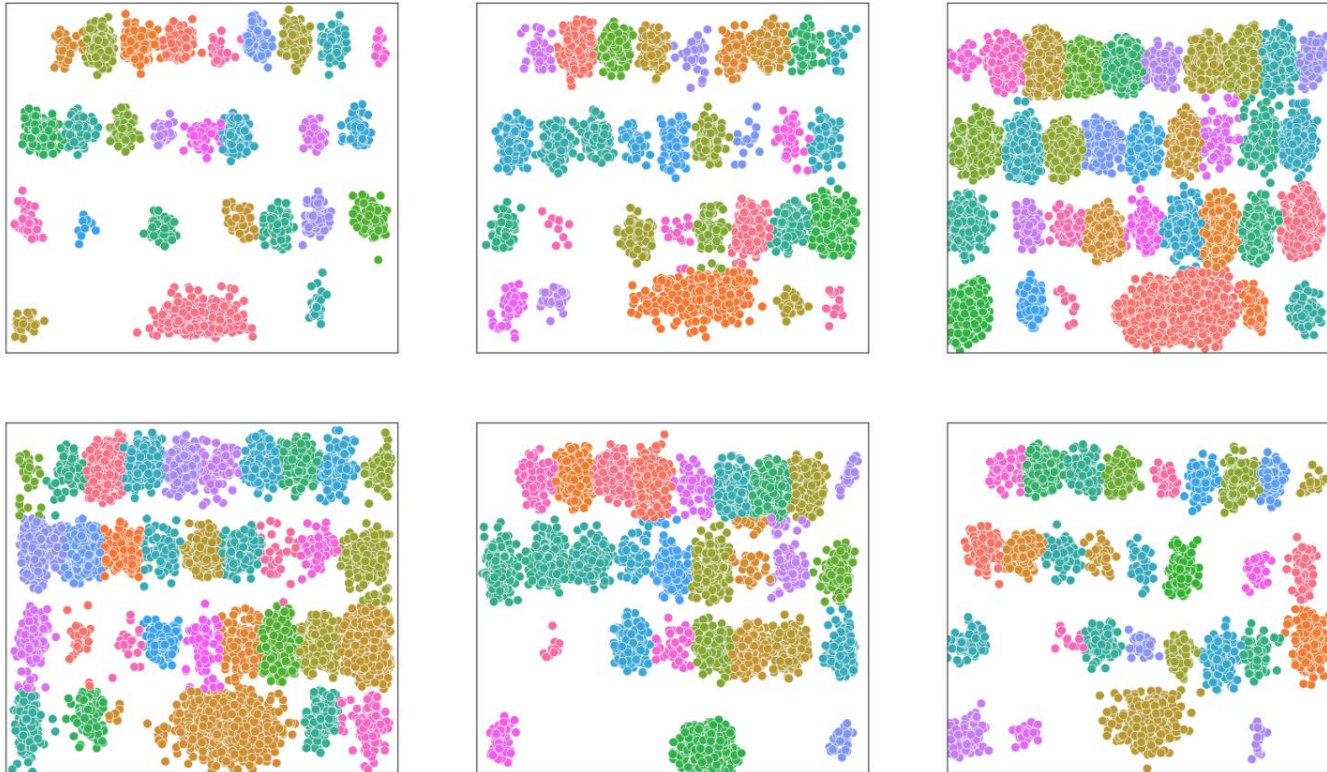
Smartphone



[Azenkot and Zhai 2012]

Individual typing behaviour

... is visible in the touch points on a keyboard



What kind of **differences** in typing and keyboard use can you spot between these people?

[Buschek et al. 2018]

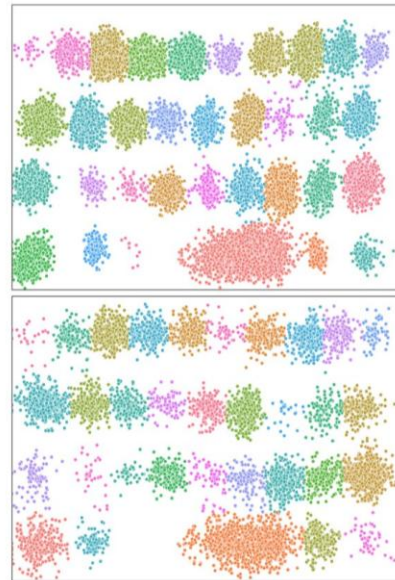
Adapting keyboards to individual users

Overview

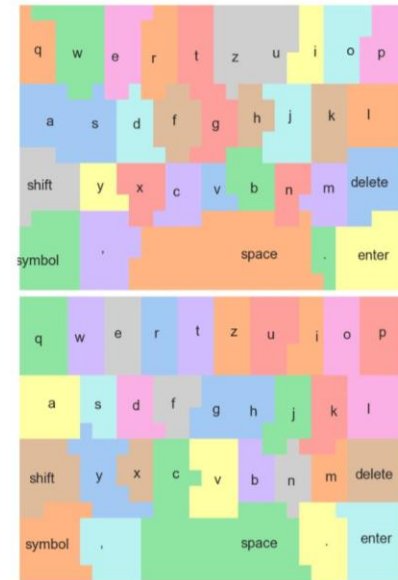
Visible keyboard



Collect touches



Adapt underlying key regions



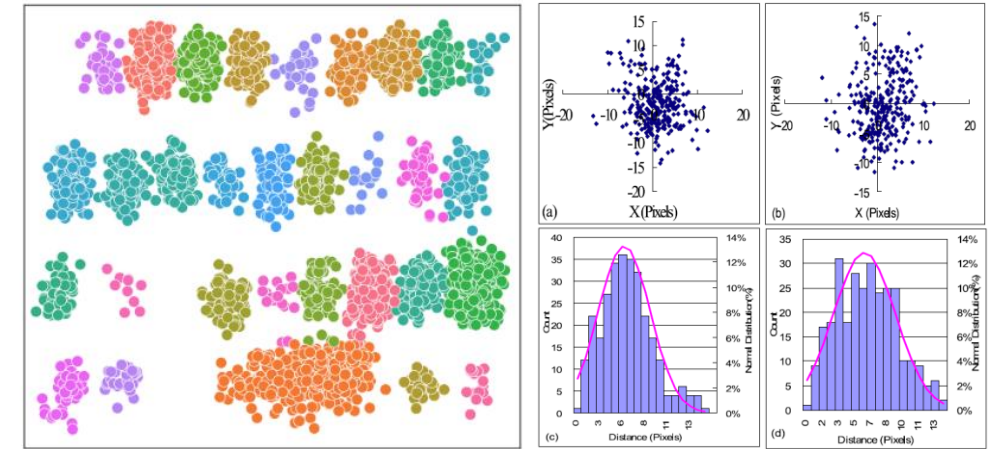
User A

User B

Modelling keypress touches

From x,y touch points to one Gaussian per key

- Observation: touch points distributed in groups around targets (= keys)
- Studies show that this variation in points around targets is approx. normally distributed
- Therefore: Use a normal distribution as a model per key

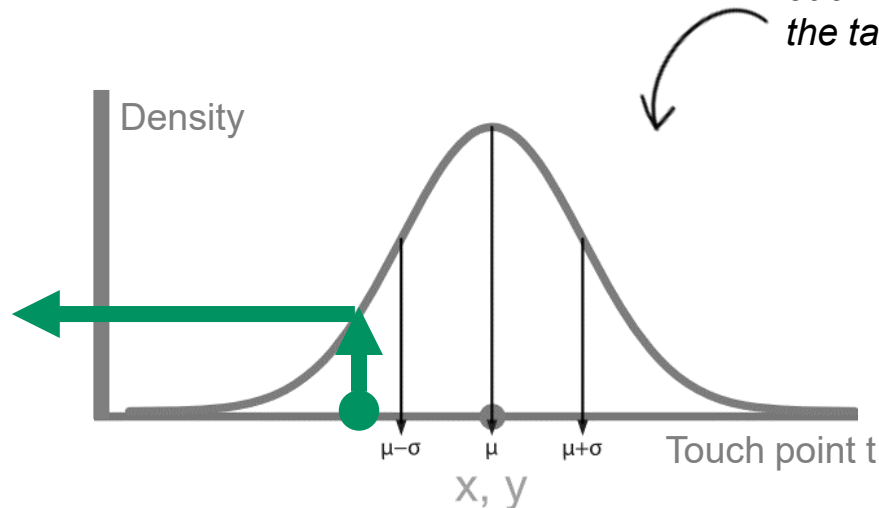


[Wang and Ren, 2009]

„The probability of touching at point t , if the user has aimed at key k , ...

$$p(t|k) = \mathcal{N}(\mu_k, \sigma_k^2)$$

... is normally distributed (with mean μ and variance σ^2)“

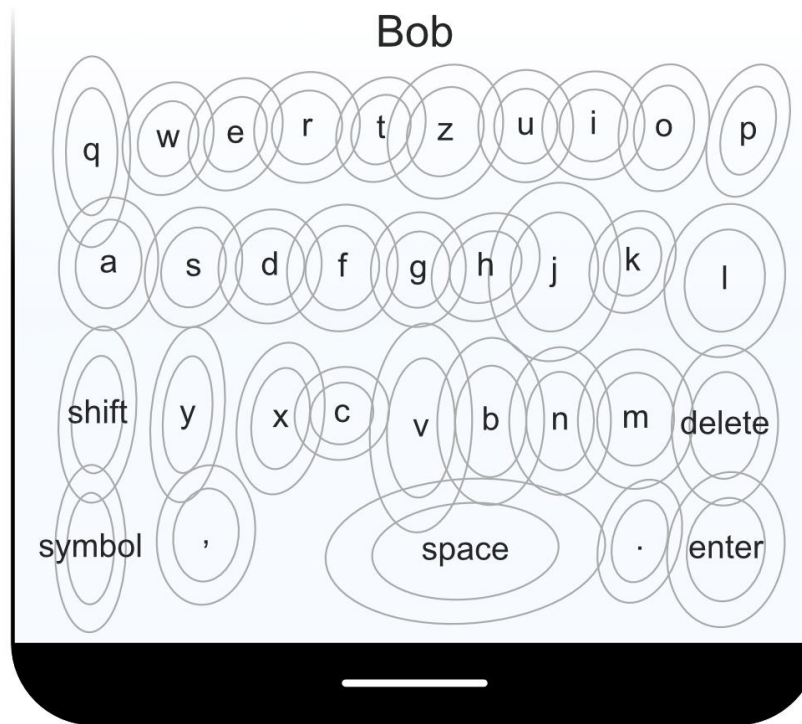
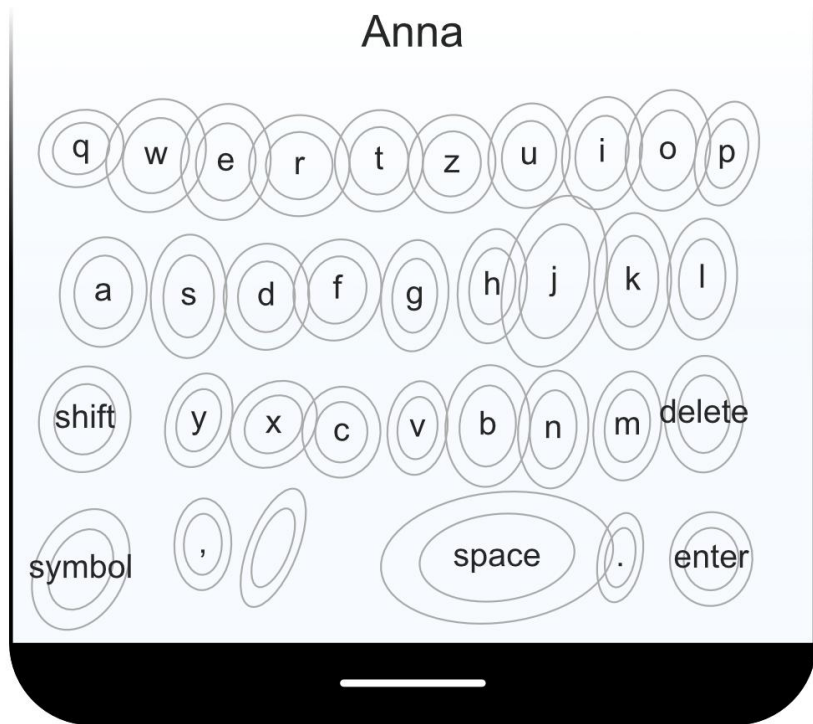


Decoding single keypresses

Keyboard model

Using normal distributions

Visualization of the motivated model (one normal distribution per key), for two users:



The circles show 2 and 3 standard deviations.

Note: In contrast to the , a normal distribution does not “end” – even touch points far away from the centre get a (tiny) positive likelihood.

Applying the keyboard model

To work out which key a user intends to activate with a given touch

- Inference / decoding: Applying the model to infer the user's intended key (or “decode the touch”)

- So far we know: $p(t|k) = \mathcal{N}(\mu_k, \sigma_k^2)$

- But we need: $p(k|t)$ „If touched at point t , how likely is it that the user wanted to hit the key k .“

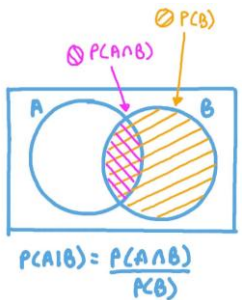
- Solution: Bayes' Rule

$$p(k|t) = \frac{p(t|k)p(k)}{p(t)}$$

Sidenote:

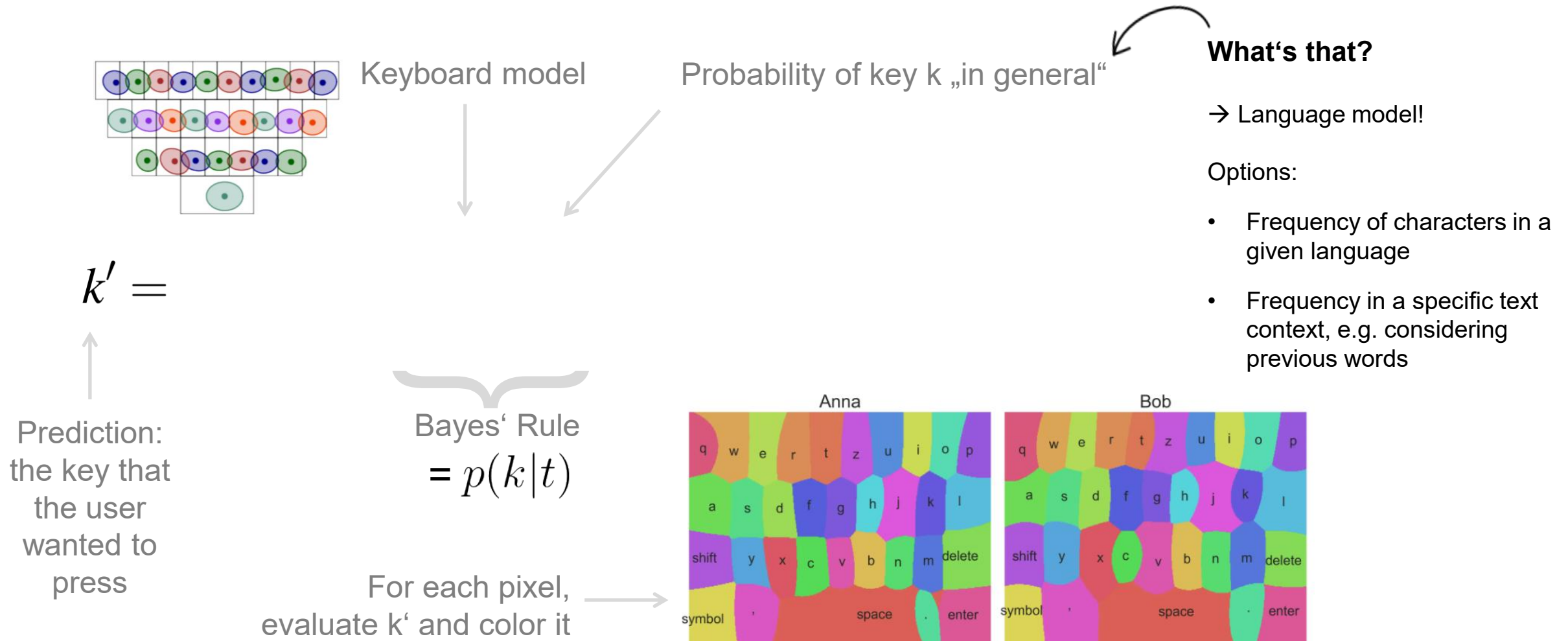
Bayes' rule follows from the rules of conditional probability:

$$p(k|t) = \frac{p(k \cap t)}{p(t)} = \frac{\frac{p(k \cap t)}{p(k)}p(k)}{p(t)} = \frac{p(t|k)p(k)}{p(t)}$$



Applying the keyboard model

To work out which key a user intends to activate with a given touch



DIY: Probabilistic keyboard model

```
touchX = ... // touch X coordinate
touchY = ... // touch Y coordinate
num_keys = ... // number of keys on keyboard
means = [...] // list of all key means (2D key locations)
variances = [...] // list of key variances (real values) or covariances (2x2 matrices)

probs = [] // list to store the likelihoods of each key being pressed
sum = 0 // variable to store sum of likelihoods for normalisation (see below)

for k = 0 to num_keys: // iterate over all keys
    // evaluate touch location under distribution of the key*:
    prob_t_given_k = multinormal_pdf(touchX, touchY, means[k], variances[k])
    // likelihood of key without touch info; uniform (here), or based on language*:
    prob_k = 1/num_keys
    // store product and add it to the sum of all likelihoods*:
    probs[k] = prob_t_given_k * prob_k
    sum = sum + probs[k]

// normalise, so that the likelihoods add up to 1*:
probs = probs / sum //note: "/" is element-wise division

// find most likely key:
pressed_key_index = argmax(probs)
// TODO for adaptation: update means and variances with new touchX and touchY
```

Activity



Check your keyboard app and its settings:

Can you find any **evidence of keyboard adaptation**?

E.g. collected touch data, settings referring to adaptation, etc.



<https://www.microsoft.com/en-us/swiftkey>

Pitfalls of adaptation

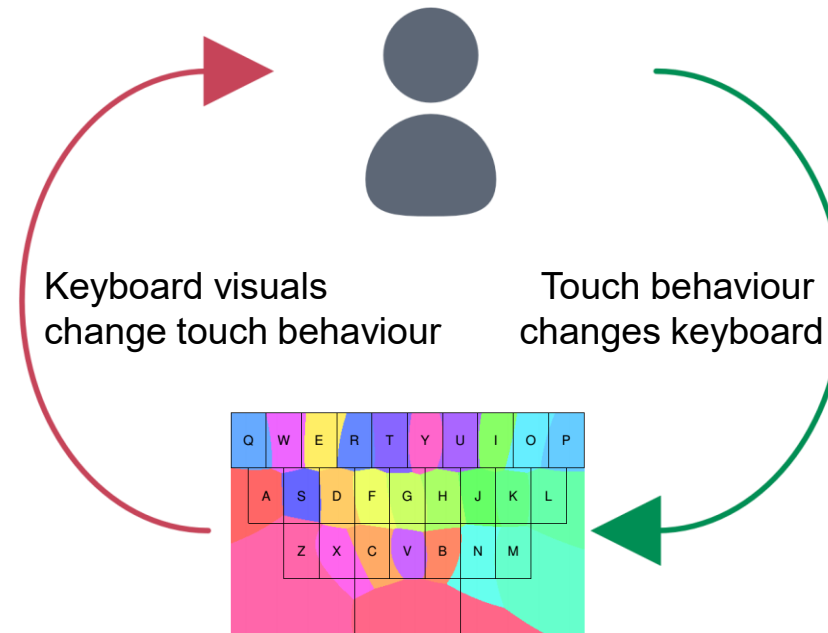
Adaptation in the background

Why do our keyboards not look like this?



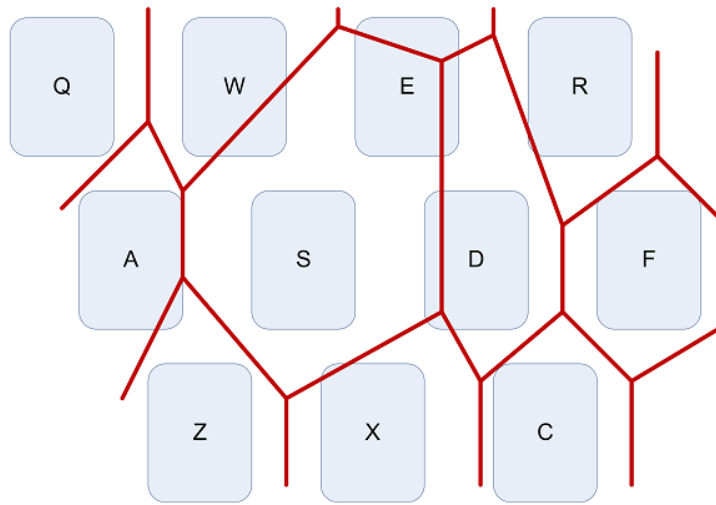
[Yin et al. 2013]

→ Avoid co-adaptation of user and system

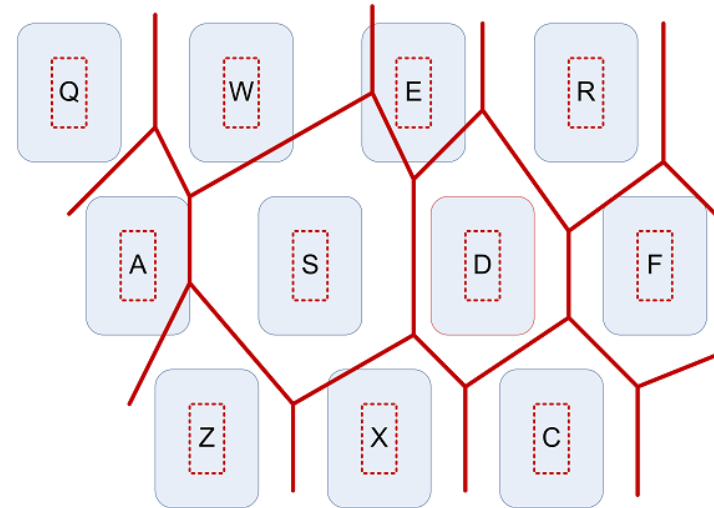


Adaptation vs distortion

Unlimited adaptation



With protected key region



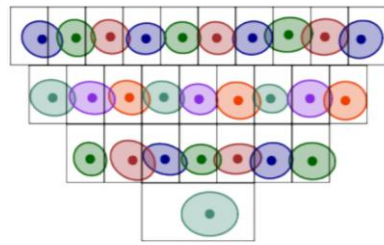
Here: (almost) impossible to type „e“!

[Gunawardana et al. 2010]

Touch + X

Integrating further information beyond keypresses

Our model so far



Keyboard model

Language model?

$$k' = \operatorname{argmax}_{k \in K} \left(\frac{p(t|k)p(k)}{p(t)} \right) = \operatorname{argmax}_{k \in K} (p(t|k)p(k))$$

Option 1: Uniform distribution

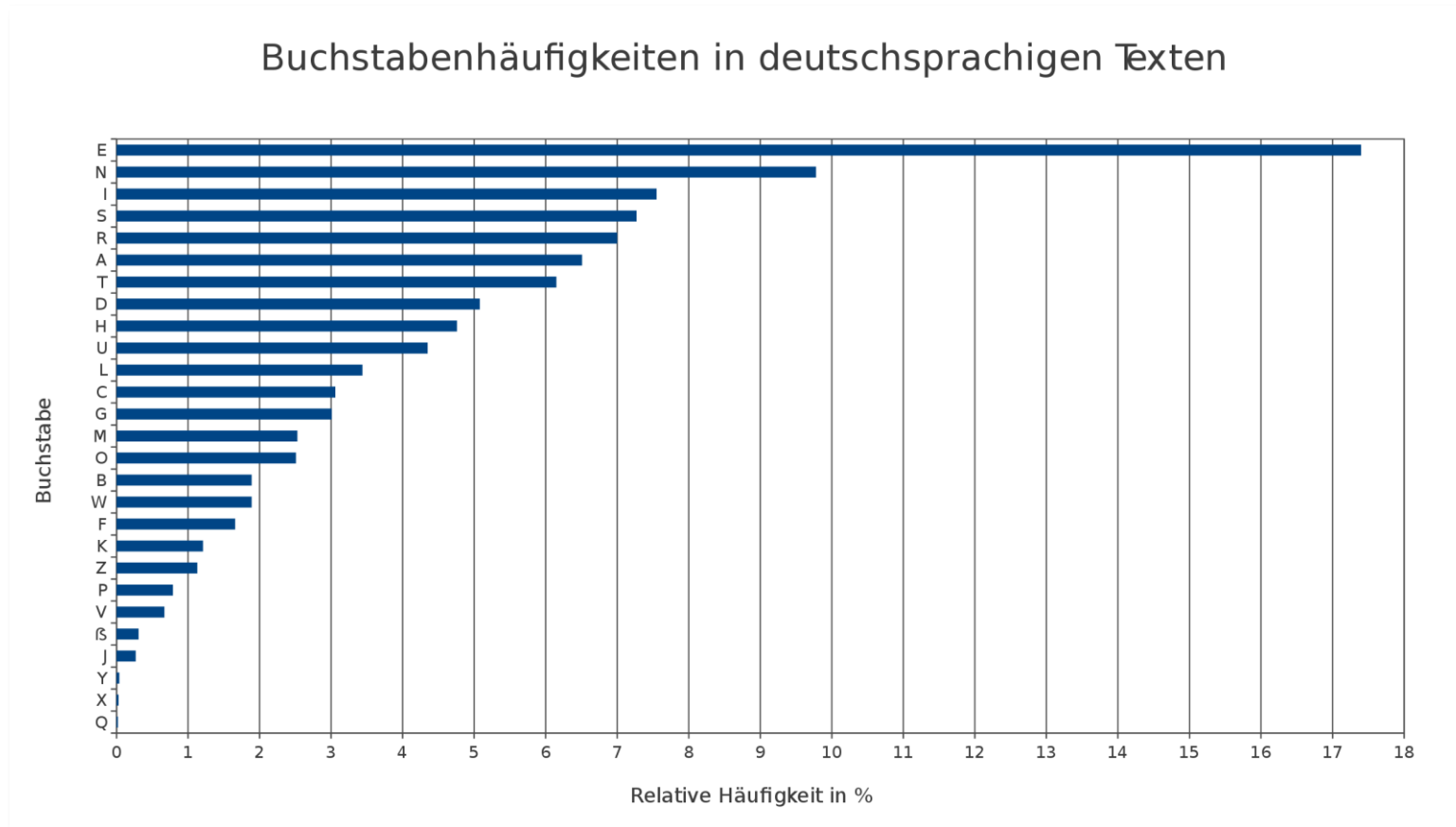
- Implements a simplified assumption:
We assume that all characters (and thus keys) are equally important for the user, all the time
- With this, the language model is

$$p(k) = \frac{1}{N} \quad \text{For all } N \text{ keys/characters}$$

Option 2: Unigram model

- Idea: Use overall frequencies of characters in a language
- With this, the language model is

$$p(k = \text{"e"}) = 0.174$$



https://de.wikipedia.org/wiki/Buchstabenh%C3%A4ufigkeit#/media/Datei:Buchstabenh%C3%A4ufigkeit_Deutsch.svg

Option 3: Bigram model

... and n-gram models in general

- Frequencies of character sequences in a language
- E.g. for English: „**th**“ is very common
- With this, the language model is

$$p(k_i = \text{“}h\text{”} | k_{i-1} = \text{“}t\text{”}) = \dots$$

Outlook :

- Longer contexts, e.g. 5 characters $p(k_i | k_{i-5} \dots k_{i-1})$
- More complex models beyond n-grams, e.g. deep learning

```
bigram, frequency
th, 100272945963
he, 86697336727
in, 68595215308
er, 57754162106
an, 55974567611
re, 52285662239
on, 49570981965
at, 41920838452
en, 41004903554
nd, 38129777631
ti, 37856196209
es, 37766388079
or, 35994097756
te, 33973261529
...
```

<https://gist.github.com/lydell/c439049abac2c9226e53#file-bigrams-json>

Combining Touch + Language

- Previously we had:

$$k' = \operatorname{argmax}_{k \in K} \left(\frac{p(t|k)p(k)}{p(t)} \right) = \operatorname{argmax}_{k \in K} (p(t|k)p(k))$$

- E.g. with our bigram model we have instead:

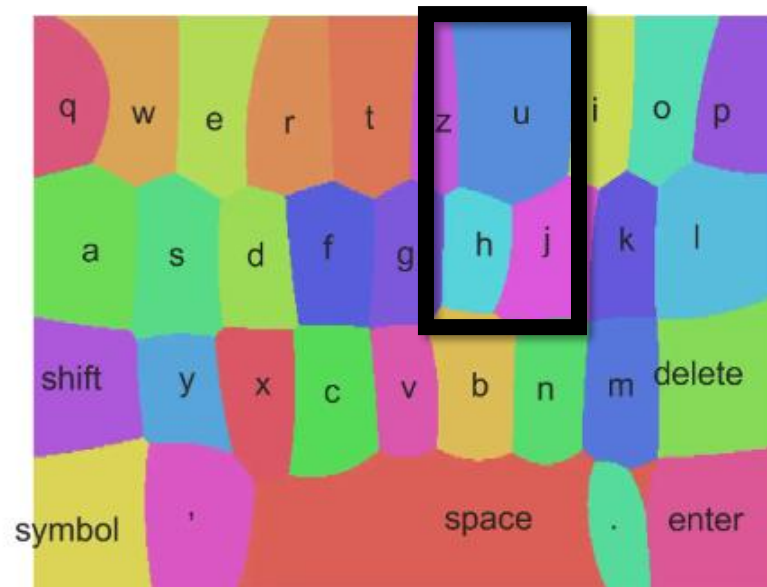
$$k'_i = \operatorname{argmax}_{k_i \in K} (p(t|k_i)p(k_i|k_{i-1}))$$

Application example

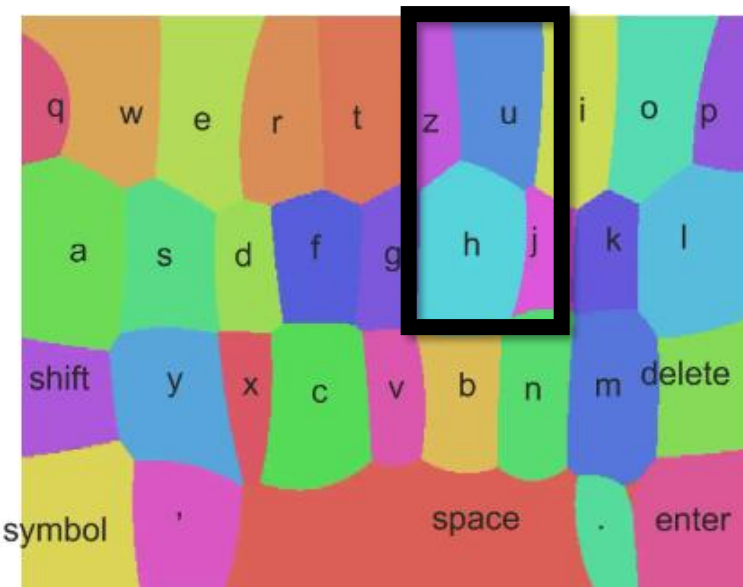
Effect of including language context

- This example: bigram model, English
- The same keyboard, after two different previous characters:

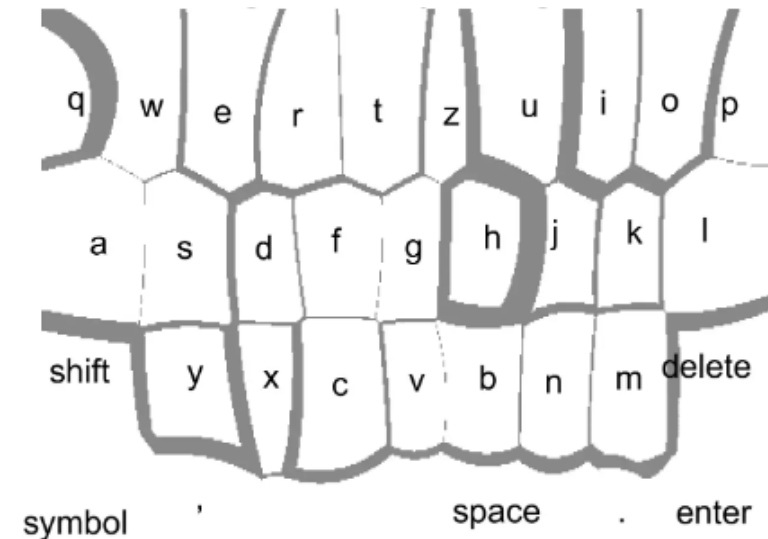
Last character was “q”



Last character was “t”



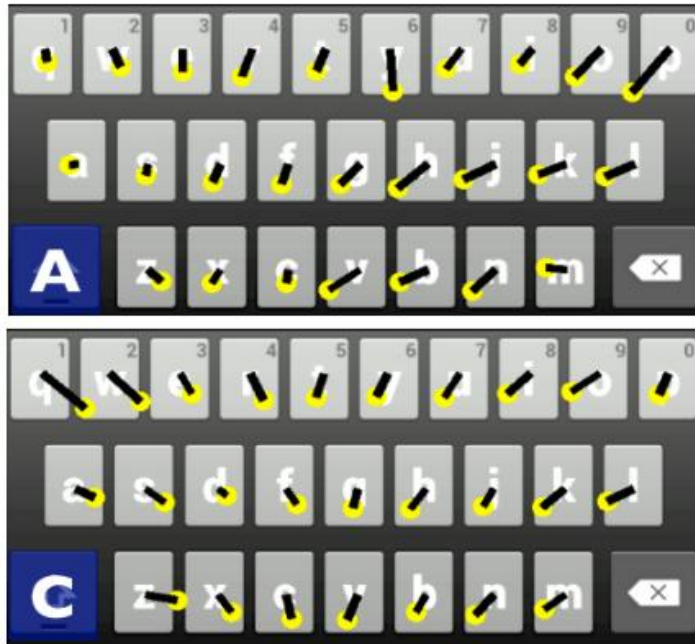
Pixel differences



Other Context Adaptations

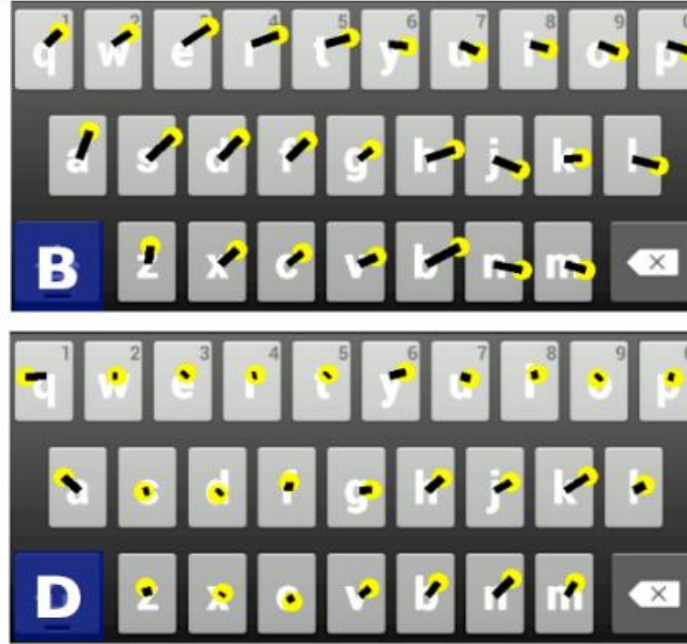
Beyond language - e.g. hand posture

Left thumb



Index finger

Right thumb

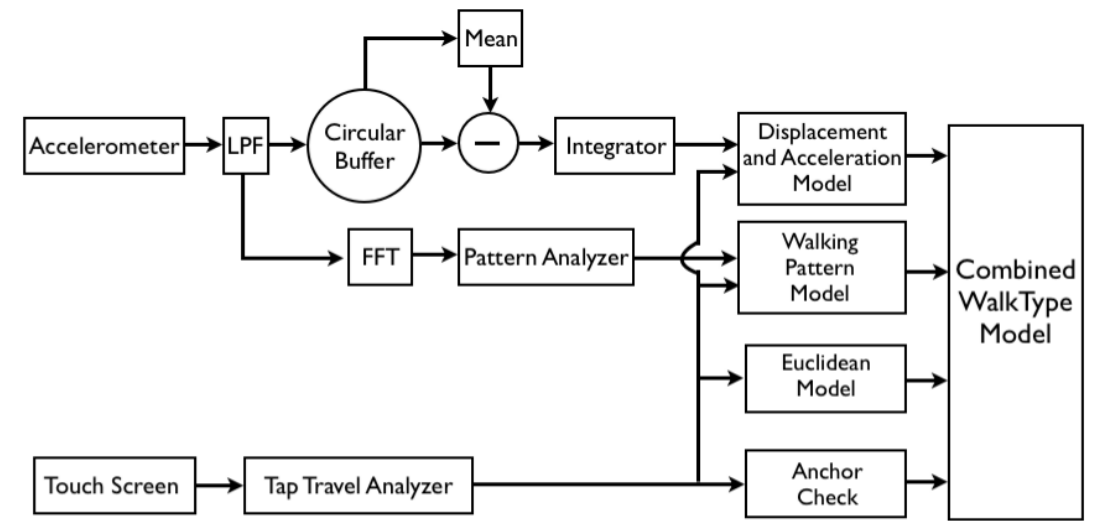


Two thumbs

[Goel et al. 2013]

Other Context Adaptations

E.g. walking



[Goel et al. 2012]

Decoding typing sequences

Motivation

Why is it useful to try and “look at” (decode) all touches of a word at once?

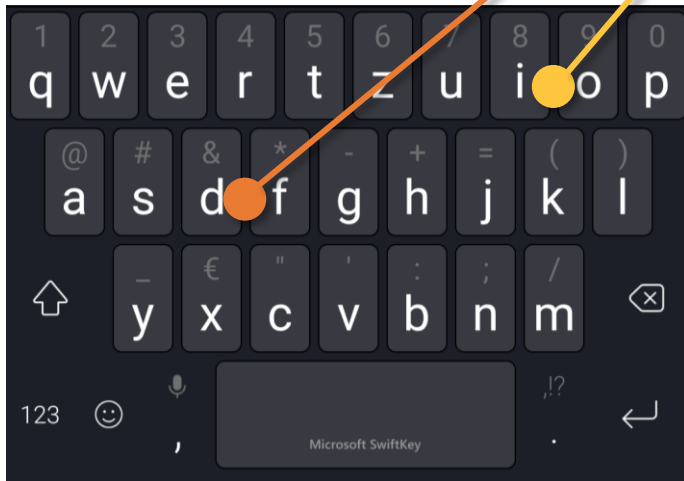
Example

„Next year, I’d like to study In[?][?] “

?

„Next year, I’d like to study **Indian history**“

„Next year, I’d like to study **Informatics**“



If we had this instead, it’s not hard to work out:

„Next year, I’d like to study In[?][?]rmatics“

→ So, if we look at a **word as a whole**, we can reduce the uncertainty of our inference about individual touch points!

But: to be able to do that, we have to change words *after* they have been entered completely → auto correct

Decoding Typing Sequences

We could even try to infer the whole sentence in one go

- Infer intended input after entering whole word or sentence
 - More evidence for inference
 - No need for user to pay attention to intermediate output
 - No intermediate feedback
- Example (sentence-based decoding):

„pleaseforwarxmetheattachement“



„Please forward me the attachment.“



[Vertanen et al. 2015]

Sidenote: The paper was called “VelociTap” since the author thought you could type as fast as a velociraptor. 😊

Sequence decoding, formally

- Sequence of user's intended keys/letters (i.e. unknown to system)

$$s = \{k_1, k_2, \dots, k_n\}$$

- Sequence of user's touches (i.e. system's observations)

$$o = \{t_1, t_2, \dots, t_m\}$$

- Can we infer s from o ?
$$s' = \operatorname{argmax}_s (p(o|s)p(s))$$

Note: Same approach as for a single touch but now we have sequences of touches and keys

$$k' = \operatorname{argmax}_{k \in K} \left(\frac{p(t|k)p(k)}{p(t)} \right) = \operatorname{argmax}_{k \in K} (p(t|k)p(k))$$

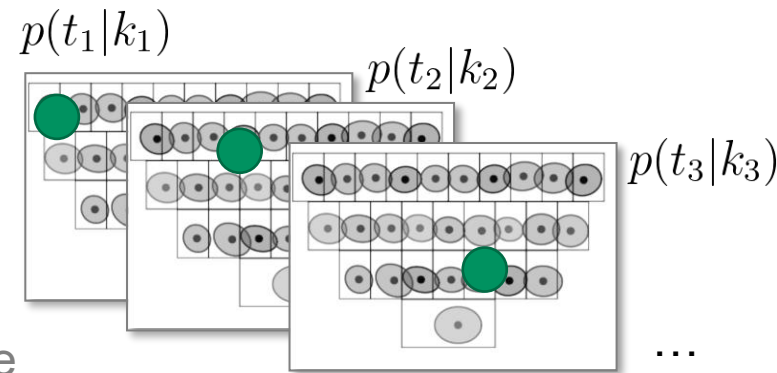
Probabilistic model for sequences

See [python notebook](#) for an example implementation

Prior over letter sequences,
i.e. probability of s in a language,
i.e. a language model

$$s' = \operatorname{argmax}_s (p(o|s)p(s))$$

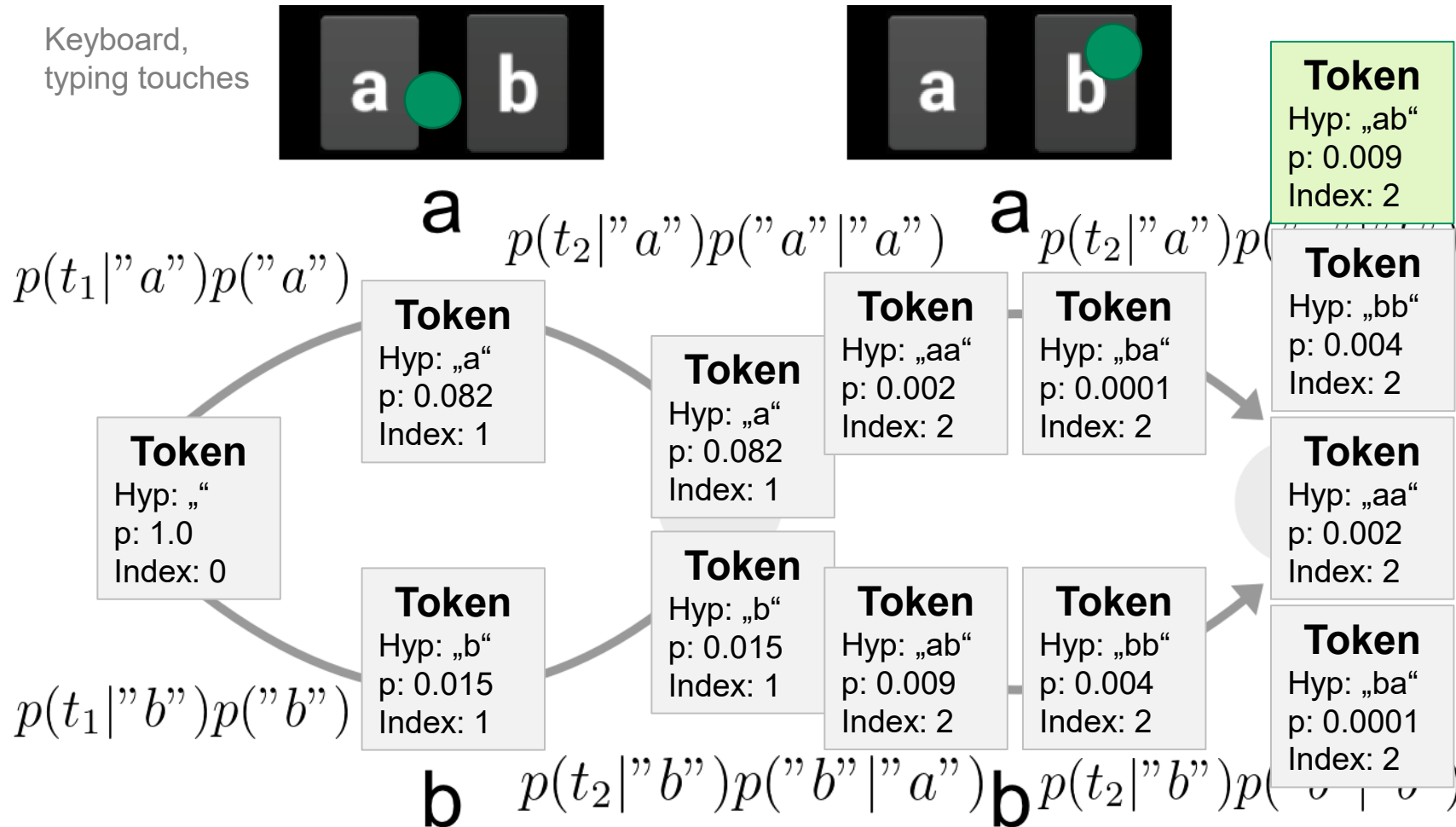
$$p(o|s) = \prod_{i=1}^m p(t_i|k_i)$$



Keyboard model, now for a touch sequence
(i.e. likelihood of observing the touch sequence o under the assumption that the user intended to write s)

Decoding typing sequences

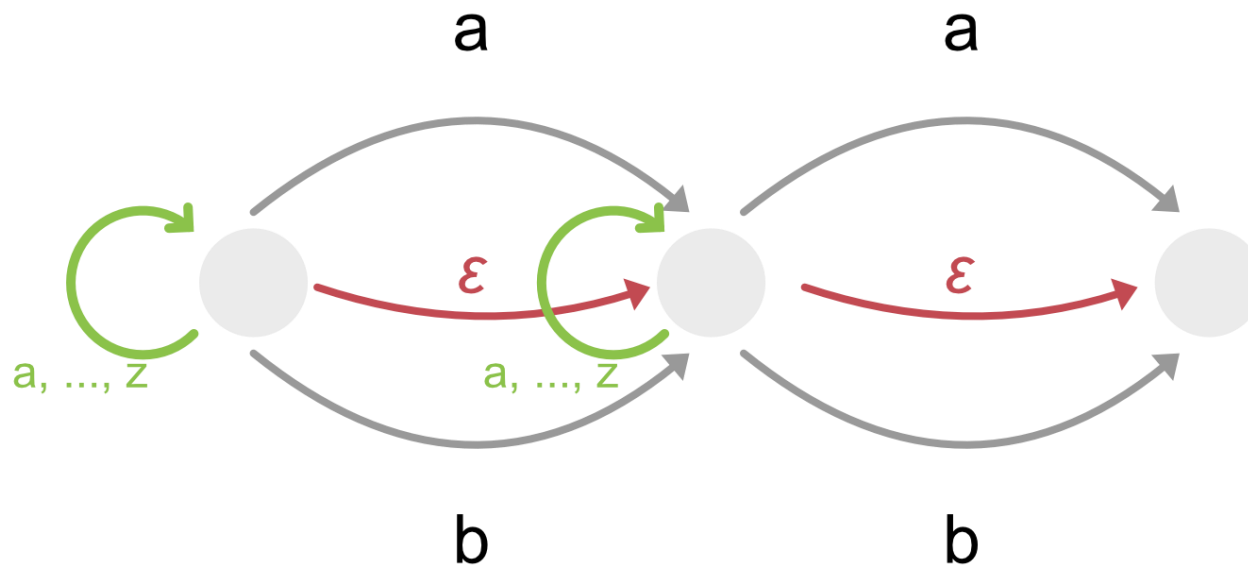
Token passing algorithm



Decoding Typing Sequences

With insertion and deletion

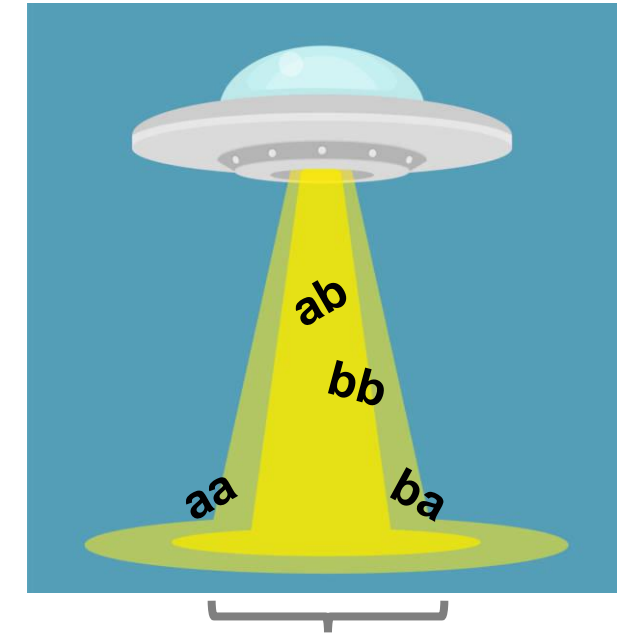
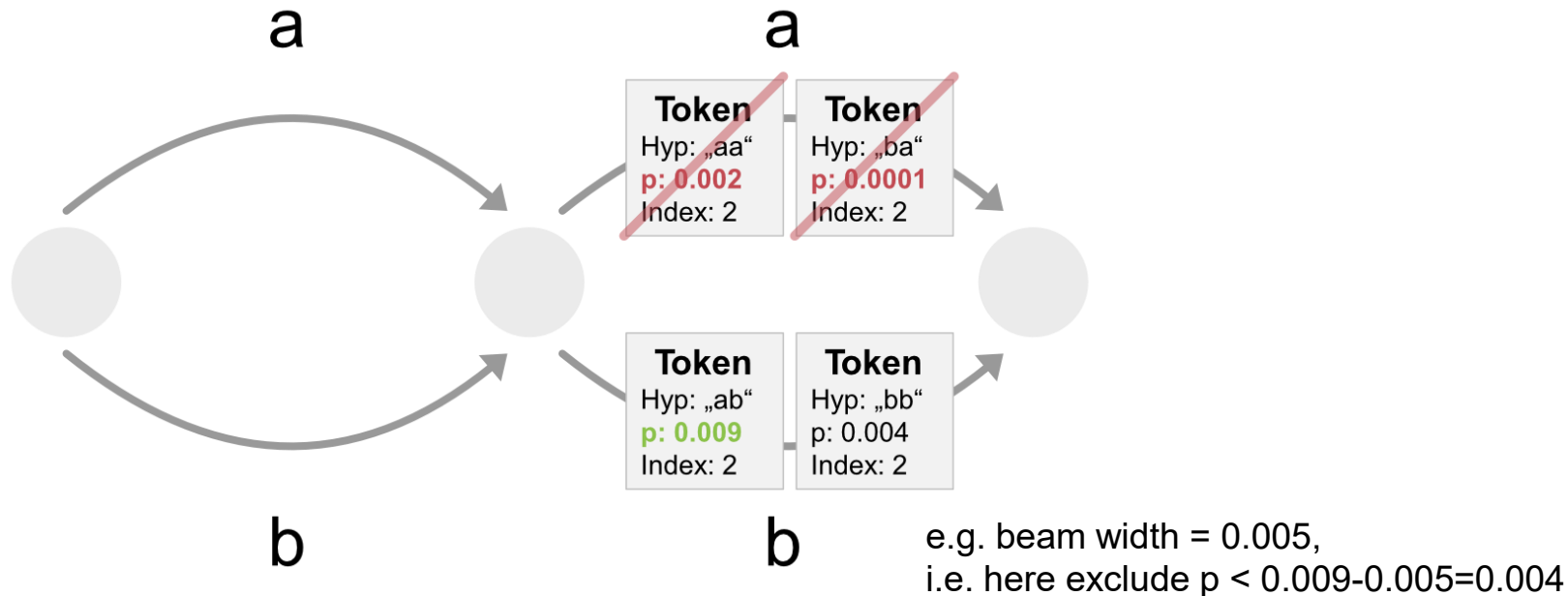
- Previous slide: Substitution-only decoder
- Extensions: **insertion** and **deletion** transitions, with „penalty“



Decoding typing sequences

With beam search / pruning

- Problem: Large search space
Substitution-only $\rightarrow$ exponential, Insertion $\rightarrow$ infinite
- Solution: **Beam search / pruning**
Per index, only propagate tokens that are within a certain range (=„beam width“) of the probability of the most likely token.



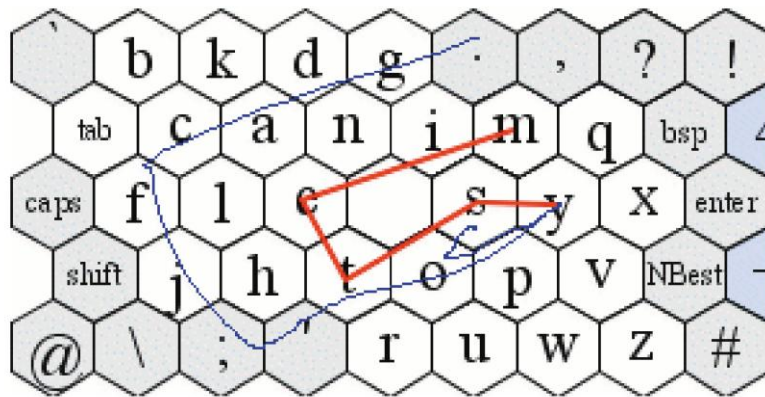
„Beam width“,
i.e. width of the „search beam“

Note: In the lecture notebook the beam width is implemented multiplicatively (i.e. „X% of the most likely token“ $\rightarrow$ smaller % value = broader beam)

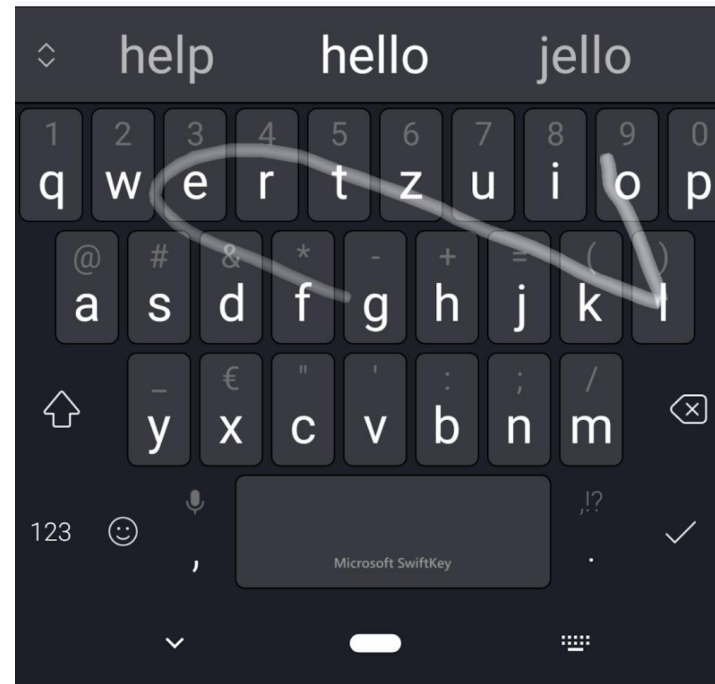
Gesture-based Decoding

Gesture-based decoding

Infer intended word from shape of finger trace on the keyboard



„SHARK2“ [Kristensson and Zhai 2004]



Microsoft SwiftKey

Gesture-based decoding

Infer intended word from shape of finger trace on the keyboard

$$w' = \operatorname{argmax}_{w \in W} (p(\text{trace} | w) p(w))$$

Shape model $\swarrow$ $\nwarrow$ Language model

Stored template (ideal) shapes
for all words in dictionary W



**Distance
metric**

e.g. see
Kristensson and Zhai,
2004

User's touch trace



Decoding without keypresses?

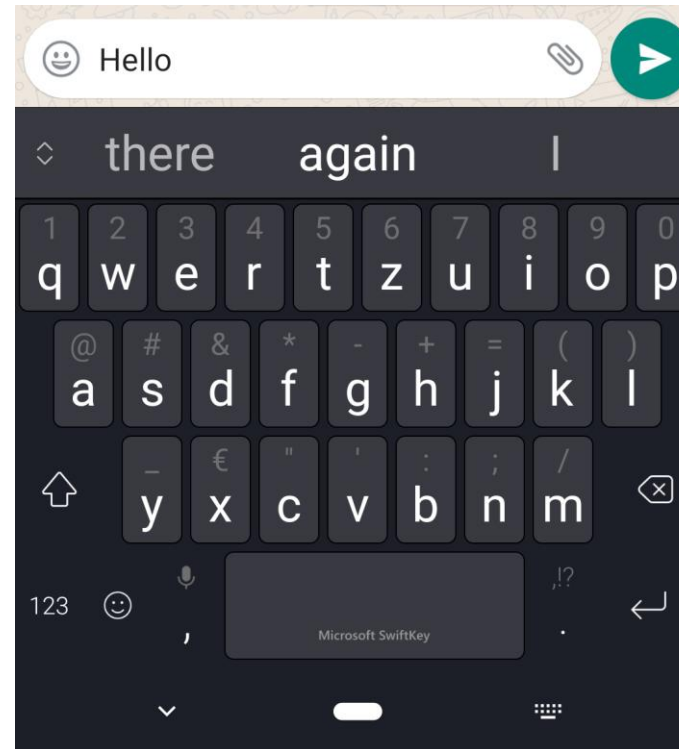
Word prediction

Or „word suggestions“

- So far: Inference used touch input
- Now: Predict *next* word that user has not yet started to type, only using language context

$$p(w_t | w_{t-n} \dots w_{t-1})$$

- E.g. n-gram word models, i.e. context of last n-1 words
- More recently: Deep Learning to include larger context



Summary

Summary

Overview

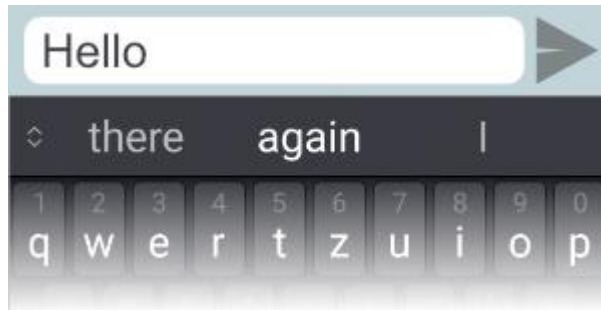
- Improving UIs (here: keyboards)
by probabilistically combining input information (here: touch)
with other information (here: language)
- **Adaptation:**
 - Individual input behaviour → adaptation to typist
 - Further sensors → adaptation to context
- **Prediction/Decoding:**
 - Single touch + language context → current key
 - Touch sequences + language context → current word/sentence
 - Language only → next word(s)

Summary

Linking this session to common “intelligent” keyboard features

Touch model + Language model,
based on touch points and large text corpus

Language model only
(no touch data needed)



Word suggestions

Decoding sequences
(after typing)



Auto-correction

Decoding „live“
character-by-character



Adapting key areas to the
individual user

Material

- Notebook on e-Learning with more visualisations and a basic token-passing implementation
- Blog post with visualisations from that notebook:
<https://towardsdatascience.com/a-look-inside-the-bayesian-keyboard-c10c53e6d5a>



Questions & Discussion



- Which problems do adaptive and predictive keyboards address?
- Explain how touch information and language information can be combined for keyboard adaptation. What effect does this achieve at the pixel level?
- Explain decoding of touch sequences with token passing and beam pruning (e.g. shortly describe the addressed problem and key steps).
- Are adaptive and predictive keyboards „deceptive“?
- Which (further) factors could be considered for adaptation and word prediction in keyboards?
- Which other UIs beyond keyboards could benefit from similar approaches? What might have to be changed?
- Create your own „exam question“ (+ answer) for this lecture:

References

References (1)

- Azenkot, S., & Zhai, S. (2012). Touch behavior with different postures on soft smartphone keyboards. *Proceedings of the 14th international conference on Human-computer interaction with mobile devices and services*, 251–260. <https://doi.org/10.1145/2371574.2371612>
- Buschek, D., Bisinger, B., & Alt, F. (2018). ResearchIME: A Mobile Keyboard Application for Studying Free Typing Behaviour in the Wild. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 1–14. <https://doi.org/10.1145/3173574.3173829>
- Findlater, L., & Wobbrock, J. (2012). Personalized input: Improving ten-finger touchscreen typing through automatic adaptation. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 815–824. <https://doi.org/10.1145/2207676.2208520>
- Goel, M., Findlater, L., & Wobbrock, J. (2012). WalkType: Using accelerometer data to accomodate situational impairments in mobile touch screen text entry. *Proceedings of the 2012 ACM Annual Conference on Human Factors in Computing Systems - CHI '12*, 2687. <https://doi.org/10.1145/2207676.2208662>
- Goel, M., Jansen, A., Mandel, T., Patel, S. N., & Wobbrock, J. O. (2013). ContextType: Using Hand Posture Information to Improve Mobile Touch Screen Text Entry. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2795–2798. <https://doi.org/10.1145/2470654.2481386>
- Goodman, J., Venolia, G., Steury, K., & Parker, C. (2002). Language modeling for soft keyboards. *Proceedings of the 7th international conference on Intelligent user interfaces*, 194–195. <https://doi.org/10.1145/502716.502753>
- Gunawardana, A., Paek, T., & Meek, C. (2010). Usability guided key-target resizing for soft keyboards. *Proceedings of the 15th international conference on Intelligent user interfaces - IUI '10*, 111. <https://doi.org/10.1145/1719970.1719986>
- Holz, C., & Baudisch, P. (2011). Understanding touch. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2501–2510. <https://doi.org/10.1145/1978942.1979308>

References (2)

- Kristensson, P. O., & Vertanen, K. (2014). The inviscid text entry rate and its application as a grand goal for mobile text entry. *Proceedings of the 16th international conference on Human-computer interaction with mobile devices & services*, 335–338. <https://doi.org/10.1145/2628363.2628405>
- Kristensson, P.-O., & Zhai, S. (2004). SHARK2: A large vocabulary shorthand writing system for pen-based computers. *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology - UIST '04*, 43. <https://doi.org/10.1145/1029632.1029640>
- Vertanen, K., Memmi, H., Emge, J., Reyal, S., & Kristensson, P. O. (2015). VelociTap: Investigating Fast Mobile Text Entry using Sentence-Based Decoding of Touchscreen Keyboard Input. *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, 659–668. <https://doi.org/10.1145/2702123.2702135>
- Yin, Y., Ouyang, T. Y., Partridge, K., & Zhai, S. (2013). Making touchscreen keyboards adaptive to keys, hand postures, and individuals: A hierarchical spatial backoff model approach. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2775–2784. <https://doi.org/10.1145/2470654.2481384>

Further Reading:

- Bi, X., Li, Y., & Zhai, S. (2013). FFitts law: Modeling finger touch with fitts' law. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1363–1372. <https://doi.org/10.1145/2470654.2466180>
- Oulasvirta, A., Kristensson, P. O., Bi, X., & Howes, A. (Hrsg.). (2018). *Computational Interaction*. Oxford University Press.